THE UNITED REPUBLIC OF TANZANIA
MZUMBE UNIVERSITY
"Tujifunze Kwa Maendeleo ya Watu"

CSS 221: Introduction to Web Programming
Assignment 3 – Web Scripting & Output Analysis

Prepared by: Bertha Msuliche Lebalwa, MSc
Faculty of Science and Technology (FST)
Department of Computing Science Studies

Academic Year 2025/2026

Instructions to Students:

- Answer all eighty (80) questions. For each question, analyze the provided code segment in JavaScript, PHP, or Perl, and write down its exact terminal / console output.
- Pay close attention to dynamic type coercions, operator behaviors, and nested loops which differ between client-side and server-side execution flows.
- If a script results in an execution error, state the specific type of Exception or Error raised and explain why.
- Submission must be compiled as a single PDF uploaded to the university LMS by the specified deadline.

Part A: JavaScript Client-Side Execution, Control Flow, and Logic (Questions 1 - 15)

Analyze the following JavaScript snippets and write the exact console output.

Question 1: TTCL Bandwidth Type Coercion

A client-side utility script processes bandwidth metrics from the Tanzania Telecommunications Corporation (TTCL) portal:

```
let basicSpeed = "45";
let boostSpeed = 5;
let promoActive = true;

let totalSpeed = basicSpeed + boostSpeed;
let finalSpeed = basicSpeed - boostSpeed;
let grandTotal = totalSpeed + promoActive;

console.log(totalSpeed + " | " + finalSpeed + " | " + grandTotal);
```

Question 2: HESLB Loan Modulo Allocation Loop

The Higher Education Students' Loans Board (HESLB) portal computes a hash validation check via loops:

```
let seed = 17;
let accumulator = 0;

for (let i = 1; i <= 5; i++) {
    if (seed % i === 0) {
        accumulator += i * 2;
    } else {
        accumulator -= 1;
    }
}
console.log(accumulator);
```

Question 3: SGR Station Alert Code Array Modification

The Standard Gauge Railway (SGR) client board replicates and slices station boarding sequences:

```
let stations = ["DAR", "MORO", "DOM", "MZA"];
stations.push("SHY");
let removed = stations.splice(1, 2, "KILOSA");

console.log(stations.join("-") + " | " + removed.join("-"));
```

Question 4: Mzumbe SIMS Verification Matrix

The Mzumbe Student Information System (SIMS) database tests student validation states:

```
let registrationCode = "050";
let systemVal = 50;

let test1 = (registrationCode == systemVal);
let test2 = (registrationCode === systemVal);
let test3 = (Number(registrationCode) === systemVal);

console.log(test1 + " | " + test2 + " | " + test3);
```

Question 5: Tigo Pesa Short-Circuit Execution Tree

A front-end balance checker parses user verification states under short-circuit logic:

```
let hasNIDA = false;
let tierLevel = 2;
let hasActiveBalance = true;

let accessPermitted = (hasNIDA || tierLevel > 1) && (hasActiveBalance || false);
let fallbackOption = hasNIDA && "OptionA" || "OptionB";

console.log(accessPermitted + " | " + fallbackOption);
```

Question 6: CRDB Card Destructuring & Rest Parameter

A security migration script parses simulated ATM parameters using modern JavaScript destructuring:

```
let cardDetails = ["CRDB-4022", "Amina Juma", "TZS 500000", "Gold"];
let [cardNumber, cardHolder, ...metaData] = cardDetails;

let [balance, tier] = metaData;
console.log(cardNumber + " | " + cardHolder + " | " + balance + " | " + tier);
```

Question 7: NHIF Policy JSON Aggregate Processing

Evaluating nested loops and processing objects dynamically extracted from an NHIF JSON API payload:

```
let jsonPayload = '{"policyId": "NHIF-901", "dependencies": ["Dependant1", "
    Dependant2"]}';
let parsed = JSON.parse(jsonPayload);
let logOutput = "";

for (let key in parsed) {
    if (Array.isArray(parsed[key])) {
        logOutput += parsed[key].length;
    } else {
        logOutput += parsed[key] + "_";
    }
}
console.log(logOutput);
```

Question 8: LATRA Bus Route Object Reference Mutations

The Land Transport Regulatory Authority (LATRA) system tracks reference identity values in memory:

```
let routeA = { origin: "Dar", destination: "Dodoma", stops: 4 };
let routeB = routeA;
routeB.stops = 6;

let routeC = Object.assign({}, routeA);
routeC.origin = "Morogoro";

console.log(routeA.stops + " | " + routeA.origin + " | " + routeC.origin);
```

Question 9: Mzumbe Library Access Variable Closures

Analyzing lexical scoping closures utilized by the automated library scanner gates:

```
function createGateTracker(gateName) {
    let entryCount = 0;
    return function() {
        entryCount += 1;
        return gateName + ":" + entryCount;
    };
}

```

```
let gate1 = createGateTracker("FST-Library");
let gate2 = createGateTracker("Main-Library");

console.log(gate1() + " | " + gate1() + " | " + gate2());
```

Question 10: TRA TIN Formatting String Splice

A tax submission script processes string slices of input Taxpayer Identification Numbers:

```
let rawTIN = "  999-888-777  ";
let cleanTIN = rawTIN.trim();
let formatted = cleanTIN.substring(0, 3) + cleanTIN.slice(-3);

console.log(cleanTIN.length + " | " + formatted + " | " + cleanTIN.indexOf("-"))
    ;
```

Question 11: NECTA Grade Mapping Filters

The National Examinations Council of Tanzania (NECTA) script executes list filtering logic on GPA indices:

```
let marks = [45, 82, 60, 90, 33];
let passed = marks.filter(m => m >= 50);
let adjusted = passed.map(m => m + 5);

console.log(passed.length + " | " + adjusted.join(","));
```

Question 12: SGR Seat Allocation Chain Ternary

Determining localized booking tiers using nested conditions:

```
let age = 62;
let loyaltyMember = true;

let tier = age < 12 ? "Child" : (age >= 60 ? (loyaltyMember ? "SGR-Gold" : "
    Senior") : "Standard");
console.log(tier);
```

Question 13: TTCL Router Search Label Break

Navigating a multi-dimensional server rack node searching for channels:

```
let matrix = [
    [10, 20],
    [30, 40],
    [50, 60]
];
let sum = 0;

outerLoop: for (let r = 0; r < matrix.length; r++) {
    for (let c = 0; c < matrix[r].length; c++) {
        if (matrix[r][c] === 40) {
            break outerLoop;
        }
```

```
        sum += matrix[r][c];
    }
}
console.log(sum);
```

Question 14: HESLB Allocation Score Comparator

A comparison callback sorts application arrays by multiple nested parameters:

```
let students = [
    { name: "Amina", score: 85 },
    { name: "Juma", score: 92 },
    { name: "Sarah", score: 85 }
];

students.sort((a, b) => {
    if (a.score === b.score) {
        return a.name.localeCompare(b.name);
    }
    return b.score - a.score;
});

console.log(students[0].name + " | " + students[1].name + " | " + students[2].
    name);
```

Question 15: Simulated DOM Node Registry Array Tracker

A client-side script parses a series of node operations:

```
let elementPool = ["div#total-price", "span#error-msg", "div#cart-counter"];
let history = [];

for (let item of elementPool) {
    let parts = item.split("#");
    history.push(parts[1].toUpperCase());
}
console.log(history.reverse().join(" -> "));
```

Part B: PHP Server-Side Loop & Structural Tracing (Questions 16 - 45)

Analyze the following PHP script segments (adapted from traditional structural tracking tests) and write the exact output.

Question 16: Accumulative Index Variable Loop (Program A Variant)

```php
<?php
$i = 6;
$j = 3;
while ($i < 9) {
    if ($i % 2 == 0) {
        echo $j . " ";
    } else {
        echo ($j * 2) . " ";
    }
    $j = $i + $j;
    $i += 1;
}
?>
```

Question 17: Range Evaluation Modulo Branching (Program B Variant)

```php
<?php
for ($i = 2; $i < 5; $i++) {
    if ($i % 2 == 0) {
        echo ($i + 1) . " ";
    } else {
        echo ($i - 1) . " ";
    }
}
?>
```

Question 18: Decrementing Range Logic Check (Program C Variant)

```php
<?php
for ($i = 3; $i > 0; $i--) {
    if ($i % 2 != 0) {
        echo $i . " ";
    } else {
        echo ($i * 2) . " ";
    }
}
?>
```

Question 19: While-Loop Inverse Multiplication (Program D Variant)

```php
<?php
$i = 3;
while ($i > 0) {
    if ($i % 2 != 0) {
```

```
        echo $i . " ";
    } else {
        echo ($i * 2) . " ";
    }
    $i--;
}
?>
```

Question 20: Cumulative Series Summation Program (N=4)

Calculates the sequence $3 + 6 + 10 + 15 + 21 + \ldots$ up to length $N = 4$.

```
<?php
$n = 4;
$sum = 0;
$current = 3;
$increment = 3;

for ($i = 1; $i <= $n; $i++) {
    $sum += $current;
    echo $current . " ";
    $current += $increment;
    $increment += 1;
}
echo "= " . $sum;
?>
```

Question 21: Twin Prime Range Structural Validation Loop

Determining values output by a dry-run check of twin primes between indices 3 and 15:

```
<?php
$low = 3;
$high = 15;

function isPrime($num) {
    if ($num < 2) return false;
    for ($d = 2; $d * $d <= $num; $d++) {
        if ($num % $d == 0) return false;
    }
    return true;
}

for ($i = $low; $i <= $high - 2; $i++) {
    if (isPrime($i) && isPrime($i + 2)) {
        echo "({$i}, " . ($i + 2) . ") ";
    }
}
?>
```

Question 22: Alternating Step Accumulator Loop

```
<?php
$total = 0;
for ($x = 10; $x <= 25; $x += 4) {
```

```
    if ($x % 3 == 0) {
        $total += $x;
    } else {
        $total -= 2;
    }
}
echo $total;
?>
```

Question 23: Double Variable Tracing Matrix

```
<?php
$a = 1;
$b = 10;
while ($a < $b) {
    $a += 2;
    $b -= 1;
    echo "{$a}-{$b} ";
}
?>
```

Question 24: Modulo Division Index Skip Loop

```
<?php
for ($index = 1; $index <= 7; $index++) {
    if ($index == 4) {
        continue;
    }
    if ($index % 3 == 0) {
        echo ($index * 10) . " ";
    } else {
        echo $index . " ";
    }
}
?>
```

Question 25: Character Conversion Frequency Loop

```
<?php
$word = "MZUMBE";
for ($i = 0; $i < strlen($word); $i++) {
    if ($i % 2 == 0) {
        echo $word[$i];
    } else {
        echo strtolower($word[$i]);
    }
}
?>
```

Question 26: Exponential Threshold Iterations

```php
<?php
$val = 1;
$count = 0;
while ($val < 50) {
    $val = $val * 2 + 3;
    $count++;
    echo "{$val} ";
}
echo "C:{$count}";
?>
```

Question 27: Multi-tiered Nested Loop Tracing

```php
<?php
for ($r = 1; $r <= 3; $r++) {
    $row_output = "";
    for ($c = 1; $c <= $r; $c++) {
        $row_output .= ($r * $c) . ",";
    }
    echo rtrim($row_output, ",") . " | ";
}
?>
```

Question 28: Division Factor Checker Sequence

```php
<?php
$number = 12;
for ($factor = 1; $factor <= $number; $factor++) {
    if ($number % $factor == 0) {
        if ($factor % 2 == 0) {
            echo "E{$factor} ";
        } else {
            echo "O{$factor} ";
        }
    }
}
?>
```

Question 29: Logical Short-Circuit Variable Evaluator

```php
<?php
$p = true;
$q = false;
$counter = 0;

for ($i = 0; $i < 5; $i++) {
    if (($p || $q) && ($i % 2 == 0)) {
        $counter += 3;
    } else {
        $counter -= 1;
    }
```

```
    $p = !$p;
}
echo $counter;
?>
```

Question 30: Variable Stepping Accumulator Matrix

```
<?php
$sum = 0;
$step = 1;
for ($val = 0; $val < 15; $val += $step) {
    $sum += $val;
    $step++;
    echo "{$val}-{$step} ";
}
echo "Sum:{$sum}";
?>
```

Question 31: Array Boundary Search Tracing

```
<?php
$data = array(12, 45, 8, 23, 19, 31);
$target_limit = 20;

foreach ($data as $num) {
    if ($num < $target_limit && $num % 2 == 0) {
        echo "Y ";
    } else {
        echo "N ";
    }
}
?>
```

Question 32: String Length Index Modulo Parser

```
<?php
$items = array("DAR", "DOM", "MWANZA", "MOSHI");
foreach ($items as $name) {
    $len = strlen($name);
    if ($len % 3 == 0) {
        echo substr($name, 0, 2) . " ";
    } else {
        echo substr($name, -2) . " ";
    }
}
?>
```

Question 33: Switch Inside Loop Stepping Check

```php
<?php
$points = 0;
for ($x = 1; $x <= 4; $x++) {
    switch ($x) {
        case 1:
            $points += 10;
            break;
        case 2:
        case 3:
            $points += 20;
        default:
            $points += 5;
    }
}
echo $points;
?>
```

Question 34: Inverse Counter Termination Matrix

```php
<?php
$k = 5;
$res = 0;
while ($k < 20) {
    if ($k % 4 == 0) {
        $res += $k;
        break;
    }
    $res += 2;
    $k += 3;
}
echo "K:{$k} R:{$res}";
?>
```

Question 35: Floating Point Integer Coercion Sequence

```php
<?php
$total_score = 0;
for ($v = 1.5; $v < 6.0; $v += 1.2) {
    $coerced = (int)$v;
    $total_score += $coerced;
    echo "{$v}:{$coerced} | ";
}
echo "Total:" . $total_score;
?>
```

Question 36: Nested List Sum and Key Tracker

```php
<?php
$matrix = [
    "A" => [2, 4],
    "B" => [1, 3, 5]
];
```

```php
foreach ($matrix as $key => $row) {
    $row_sum = 0;
    foreach ($row as $val) {
        $row_sum += $val;
    }
    echo "{$key}:{$row_sum} ";
}
?>
```

Question 37: Index and Array element multiplication

```php
<?php
$numbers = [10, 20, 30];
for ($i = 0; $i < count($numbers); $i++) {
    $numbers[$i] = $numbers[$i] * $i;
}
echo implode("-", $numbers);
?>
```

Question 38: Bitwise AND Stepping in Loops

```php
<?php
$accum = 0;
for ($i = 1; $i <= 6; $i++) {
    if (($i & 1) == 0) {
        $accum += $i;
    } else {
        $accum -= $i;
    }
}
echo $accum;
?>
```

Question 39: Multidimensional Array Reference Modification Loop

```php
<?php
$grids = [
    [10, 20],
    [30, 40]
];
for ($row = 0; $row < count($grids); $row++) {
    for ($col = 0; $col < count($grids[$row]); $col++) {
        if ($row == $col) {
            $grids[$row][$col] += 5;
        }
    }
}
echo $grids[0][0] . " | " . $grids[1][1];
?>
```

Question 40: Variable Scope and Counter Mutation Check

```php
<?php
$limit = 12;
function mutateCounter() {
    global $limit;
    $limit += 8;
    return $limit;
}
for ($i = 0; $i < 2; $i++) {
    mutateCounter();
}
echo $limit;
?>
```

Question 41: String character offset shift (Simple Caesar simulation)

```php
<?php
$input_text = "ABC";
for ($i = 0; $i < strlen($input_text); $i++) {
    $ascii = ord($input_text[$i]);
    echo chr($ascii + 2);
}
?>
```

Question 42: Search Index Position Finder

```php
<?php
$names = ["Amina", "Juma", "Sarah", "Hamisi"];
$target = "Sarah";
$found_at = -1;

for ($idx = 0; $idx < count($names); $idx++) {
    if ($names[$idx] === $target) {
        $found_at = $idx;
        break;
    }
}
echo "Found:{$found_at}";
?>
```

Question 43: Cumulative sum with dynamic early exits

```php
<?php
$total_limit = 0;
for ($val = 1; $val <= 100; $val++) {
    $total_limit += $val;
    if ($total_limit > 15) {
        echo "ExitedAt:{$val} ";
        break;
    }
}
echo "Total:{$total_limit}";
```

```
?>
```

Question 44: Row division padding calculation

```php
<?php
$columns_per_row = 3;
$total_elements = 8;
$rows_needed = ceil($total_elements / $columns_per_row);

for ($r = 0; $r < $rows_needed; $r++) {
    $count = 0;
    for ($c = 0; $c < $columns_per_row; $c++) {
        $element_idx = $r * $columns_per_row + $c;
        if ($element_idx < $total_elements) {
            $count++;
        }
    }
    echo "Row{$r}:{$count} ";
}
?>
```

Question 45: Fibonacci Sequence Generation Loop

Generates the first five elements of the sequence:

```php
<?php
$prev1 = 0;
$prev2 = 1;
echo "{$prev1} {$prev2} ";

for ($i = 3; $i <= 5; $i++) {
    $current = $prev1 + $prev2;
    echo "{$current} ";
    $prev1 = $prev2;
    $prev2 = $current;
}
?>
```

Part C: PHP Advanced Server-Side Operations (Questions 46 - 60)

Analyze the following PHP script segments and write the exact output.

Question 46: CRDB Account String Concatenation vs Addition

Evaluating dynamic type boundaries using PHP operators compared to client-side behaviors:

```php
<?php
$balance = "150000";
$deposit = 50000;
$currency = " TZS";

$sumVal = $balance + $deposit;
$concatVal = $balance . $deposit . $currency;

echo $sumVal . " | " . $concatVal;
?>
```

Question 47: TRA TIN Formatter Sanitizer

The Tanzania Revenue Authority (TRA) sanitizes inputs via core string operations:

```php
<?php
$rawTIN = "  999-888-777-TRA  ";
$trimmed = trim($rawTIN);
$clean = str_replace("-", "", $trimmed);
$finalSegment = substr($clean, -6, 3);

echo strlen($trimmed) . " | " . $clean . " | " . $finalSegment;
?>
```

Question 48: LATRA Bus Route Indexed Array Pointers

Tracing mutations in list parameters when adding, merging, and popping array records:

```php
<?php
$routes = array("Dar-Moro", "Moro-Dodoma");
array_push($routes, "Dodoma-Singida");
$removed = array_pop($routes);

$extra = array("Mwanza-Shinyanga");
$final_routes = array_merge($routes, $extra);

echo count($final_routes) . " | " . $final_routes[1] . " | " . $removed;
?>
```

Question 49: Mzumbe SIMS Associative Array Keys

Verifying structural query properties on database models inside associative arrays:

```php
<?php
$student_record = array(
    "name" => "Amina Juma",
    "gpa" => 3.8,
```

```
    "registration" => "MU/FST/2025"
);

$has_gpa = isset($student_record["gpa"]);
$has_course = array_key_exists("course", $student_record);

$student_record["course"] = "BSc Computing Science";
echo $has_gpa . " | " . $has_course . " | " . $student_record["course"];
?>
```

Question 50: Tigo Pesa Foreach Reference Traps

Analyzing critical pointers and internal reference states when iterating over finance logs:

```
<?php
$transactions = array(1000, 2000, 3000);

foreach ($transactions as &$tx) {
    $tx = $tx * 1.10;
}
unset($tx); // Prevent pointer leakage check

$sum = 0;
foreach ($transactions as $tx) {
    // Checking trailing mutation side effects if reference is kept active
    $sum += $tx;
}
echo implode("-", $transactions) . " | " . $sum;
?>
```

Question 51: HESLB Allocation Arithmetic Operators

PHP evaluates modulo calculations on floating points and negative numbers. Note the output:

```
<?php
$total_batch = -17;
$divisor = 5;

$result1 = $total_batch % $divisor;
$result2 = fmod(-17.5, 5);

echo $result1 . " | " . $result2;
?>
```

Question 52: NHIF Contribution Tier Reductions

Evaluating dynamic collections mapping using standard PHP array utilities:

```
<?php
$contributions = array(15000, 22000, 8000, 35000);

$filtered = array_filter($contributions, function($amount) {
    return $amount > 10000;
});

$mapped = array_map(function($amount) {
```

```
    return $amount - 2000;
}, $filtered);

echo count($filtered) . " | " . array_sum($mapped);
?>
```

Question 53: SGR Station Waitlist Pass-By-Reference Function

Tracing parameter manipulation and references inside custom defined subroutines:

```
<?php
function processTicket($seatPrice, &$passengerName) {
    $seatPrice += 5000;
    $passengerName = "VIP " . $passengerName;
    return $seatPrice;
}

$price = 15000;
$name = "Sarah Juma";
$finalPrice = processTicket($price, $name);

echo $price . " | " . $name . " | " . $finalPrice;
?>
```

Question 54: Mzumbe Portal Session Simulation Tracker

A background script simulates a user state inside continuous system sessions:

```
<?php
// Mocking dynamic session states
$_SESSION['user'] = "Bertha";
$_SESSION['role'] = "Administrator";

if (isset($_SESSION['user'])) {
    $_SESSION['login_time'] = time();
    $status = "Active";
}

unset($_SESSION['role']);
echo $status . " | " . $_SESSION['user'] . " | " . (isset($_SESSION['role']) ? '
    Yes' : 'No');
?>
```

Question 55: TTCL Routing Multi-Dimensional Array Parsing

An engineer navigates routing coordinates over dynamic array levels:

```
<?php
$network_grid = array(
    "Zone-A" => array("Node1" => "Active", "Node2" => "Offline"),
    "Zone-B" => array("Node3" => "Active", "Node4" => "Active")
);

$active_count = 0;
foreach ($network_grid as $zone => $nodes) {
    foreach ($nodes as $node_name => $status) {
```

```
        if ($status === "Active") {
            $active_count++;
        }
    }
}
echo $active_count . " | " . $network_grid["Zone-B"]["Node3"];
?>
```

Question 56: TRA Floating Point Precision Verification Check

Determining evaluation precision when checking tax calculation margins in PHP:

```
<?php
$a = 0.1;
$b = 0.2;
$target = 0.3;

$check = ($a + $b == $target);
$formattedCheck = (abs(($a + $b) - $target) < 0.00001);

echo ($check ? "Equal" : "NotEqual") . " | " . ($formattedCheck ? "Valid" : "
    Invalid");
?>
```

Question 57: Tigo Pesa Fallthrough Switch System

A backend billing engine processes API response flags missing crucial boundaries:

```
<?php
$apiResponseCode = 2;
$feeAccumulator = 1000;

switch ($apiResponseCode) {
    case 1:
        $feeAccumulator += 500;
    case 2:
        $feeAccumulator += 1500;
    case 3:
        $feeAccumulator += 2000;
        break;
    default:
        $feeAccumulator += 5000;
}
echo $feeAccumulator;
?>
```

Question 58: NHIF Dynamic Policy Generator Closures

Evaluating scope bindings when employing anonymous closures mapping local variables:

```
<?php
$base_tax = 1500;

$calculate_premium = function($member_age) use ($base_tax) {
    if ($member_age > 50) {
        return $base_tax * 2;
```

```
    }
    return $base_tax + 500;
};

echo $calculate_premium(35) . " | " . $calculate_premium(60);
?>
```

Question 59: CRDB ATM CSV Ledger Processing

Tracing file lines parser operations on account records:

```
<?php
$csv_data = "CRDB-101,Amina,50000;CRDB-102,John,25000;CRDB-103,Sarah,90000";
$rows = explode(";", $csv_data);
$total_ledger = 0;

foreach ($rows as $row) {
    $fields = explode(",", $row);
    if (intval($fields[2]) > 30000) {
        $total_ledger += intval($fields[2]);
    }
}
echo $total_ledger;
?>
```

Question 60: LATRA Permit Verification Ternary Pipeline

Evaluating multidimensional constraints inside unified ternary statements:

```
<?php
$licenseYears = 3;
$routeAccess = "Regional";

$result = ($licenseYears > 5) ? (($routeAccess === "National") ? "Tier-1" : "
    Tier-2") : (($routeAccess === "Regional") ? "Tier-3" : "Tier-4");
echo $result;
?>
```

Part D: PHP & MySQL Database Tracing & State Operations (Questions 61 - 70)

Analyze the following script segments. These trace SQL structures, database parameters, and server states.

Question 61: PDO Fetch Array Simulation

Simulating the internal extraction flow of account records fetched using standard database models:

```php
<?php
// Mocking data fetched from "SELECT name, balance FROM Accounts"
$fetched_rows = array(
    array("name" => "Amina Juma", 0 => "Amina Juma", "balance" => 150000, 1 => 150000),
    array("name" => "Juma Hamisi", 0 => "Juma Hamisi", "balance" => 300000, 1 => 300000)
);

$output_string = "";
foreach ($fetched_rows as $row) {
    // Emulating fetching with PDO::FETCH_ASSOC
    $output_string .= $row["name"] . "_" . $row[1] . " | ";
}
echo trim($output_string, " | ");
?>
```

Question 62: SGR Seat Matrix Database Tracker

A script updates seat arrays and logs system performance metrics:

```php
<?php
$seat_bookings = "A1-Occupied,A2-Empty,B1-Occupied,B2-Occupied";
$seats_list = explode(",", $seat_bookings);
$occupied_count = 0;

for ($i = 0; $i < count($seats_list); $i++) {
    $pair = explode("-", $seats_list[$i]);
    if ($pair[1] === "Occupied") {
        $occupied_count += ($i + 1) * 10;
    }
}
echo $occupied_count;
?>
```

Question 63: HESLB Priority Checker with Exception Handling

Tracing program state shifts when execution paths throw custom exceptions:

```php
<?php
function verifyGPA($gpa) {
    if ($gpa < 0.0 || $gpa > 5.0) {
        throw new InvalidArgumentException("RangeError");
    }
```

```
    if ($gpa < 2.0) {
        throw new Exception("LowPriority");
    }
    return "Eligible";
}

$state_log = "";
try {
    $state_log .= verifyGPA(4.5) . "-";
    $state_log .= verifyGPA(1.5) . "-";
} catch (InvalidArgumentException $e) {
    $state_log .= "Invalid-";
} catch (Exception $e) {
    $state_log .= "CatchLow-";
} finally {
    $state_log .= "Done";
}
echo $state_log;
?>
```

Question 64: Mzumbe SIMS Grade JSON Constructor

A script constructs and serializes a grade matrix for a dynamic exam result portal:

```
<?php
$results = array(
    array("subject" => "CSS 221", "grade" => "A"),
    array("subject" => "CSS 223", "grade" => "B")
);

$output = array();
foreach ($results as $item) {
    $output[$item["subject"]] = ($item["grade"] === "A") ? 4.0 : 3.0;
}
echo json_encode($output);
?>
```

Question 65: Tigo Pesa Weighted Charge Calculation Loop

Calculating total service charges across multi-tier transactions with nested increment systems:

```
<?php
$transaction_payloads = array("15000_TZS", "8500_TZS", "45000_TZS");
$total_charge = 0;
$step_increment = 50;

foreach ($transaction_payloads as $tx) {
    $amount = intval(explode("_", $tx)[0]);
    if ($amount < 10000) {
        $total_charge += $amount * 0.01 + $step_increment;
    } else {
        $total_charge += $amount * 0.02 + ($step_increment * 2);
    }
    $step_increment += 25;
}
echo $total_charge;
?>
```

Question 66: CRDB Ledger Records Multi-criteria Sorting

Using anonymous comparator parameters to sort nested records:

```php
<?php
$accounts = array(
    array("id" => "A1", "gpa" => 3.5, "balance" => 50000),
    array("id" => "A2", "gpa" => 4.2, "balance" => 15000),
    array("id" => "A3", "gpa" => 3.5, "balance" => 100000)
);

usort($accounts, function($a, $b) {
    if ($a["gpa"] === $b["gpa"]) {
        return $b["balance"] - $a["balance"];
    }
    return ($b["gpa"] < $a["gpa"]) ? -1 : 1;
});

echo $accounts[0]["id"] . " | " . $accounts[1]["id"] . " | " . $accounts[2]["id"
    ];
?>
```

Question 67: TRA Asset Depreciation Loop Sequence

Tracing dynamic property updates across consecutive processing loops:

```php
<?php
$asset_value = 10000;
$depreciation_rate = 0.10;
$years = 3;

$acc_depreciation = 0;
for ($y = 1; $y <= $years; $y++) {
    $loss = $asset_value * $depreciation_rate;
    $asset_value -= $loss;
    $acc_depreciation += $loss;
}
echo round($asset_value) . " | " . round($acc_depreciation);
?>
```

Question 68: SGR Station Stop Matrix Manipulation

Navigating through internal sequential list structures using PHP pointer methods:

```php
<?php
$stops = array("Dar es Salaam", "Morogoro", "Kilosa", "Dodoma");

echo current($stops) . " | ";
echo next($stops) . " | ";
echo end($stops) . " | ";
echo prev($stops);
?>
```

Question 69: NHIF Deductions Dynamic Variable Scopes (Static Parameters)

Evaluating how values are retained inside static function call scopes:

```php
<?php
function processDeduction($amount) {
    static $accumulated_deductions = 0;
    $accumulated_deductions += $amount;
    return $accumulated_deductions;
}

processDeduction(5000);
processDeduction(12000);
$final_state = processDeduction(3000);

echo $final_state;
?>
```

Question 70: HESLB Priority Queue Array Splice Insertion

Executing structural alterations on multi-tiered arrays:

```php
<?php
$queue = array("HESLB-A", "HESLB-B", "HESLB-C");
array_splice($queue, 1, 1, array("HESLB-D", "HESLB-E"));

echo implode("-", $queue);
?>
```

Part E: Legacy Server Technologies: Perl CGI Scripting Outputs (Questions 71 - 80)

Analyze the following Perl code segments. Note sigil operations ($, @, %) and local rules.

Question 71: TTCL Diagnostic CGI Standard Scalar Declaration

A CGI script prints status headers and resolves dynamic values using basic scalars:

```perl
#!/usr/bin/perl
use strict;
use warnings;

my $status = "System Online";
my $code = 200;

print "Content-type: text/plain\n\n";
print "STATUS: $status | CODE: $code\n";
```

Question 72: Mzumbe Library CGI Perl Array Operations

Manipulating ordered lists using index arrays and basic list functions in Perl:

```perl
#!/usr/bin/perl
use strict;
use warnings;

my @books = ("Python", "XHTML", "CSS");
push(@books, "Perl");
my $popped = pop(@books);
my $shifted = shift(@books);

print "Content-type: text/plain\n\n";
print "REMAINING: ", scalar(@books), " | POPPED: $popped | SHIFTED: $shifted\n";
```

Question 73: SGR Passenger Registry Perl Hash Iteration

Resolving elements and retrieving indices inside nested hash structures:

```perl
#!/usr/bin/perl
use strict;
use warnings;

my %passenger_record = (
    "id" => "SGR-902",
    "name" => "Amina Juma",
    "class" => "VIP"
);

$passenger_record{"status"} = "Confirmed";
my $key_count = keys %passenger_record;

print "Content-type: text/plain\n\n";
print "KEYS: $key_count | CLASS: $passenger_record{'class'}\n";
```

Question 74: TRA Customs Interpolation Escapes

Tracing dynamic outputs under double-quoted vs single-quoted string declarations:

```
#!/usr/bin/perl
use strict;
use warnings;

my $tax_rate = 0.18;
my $double_quote = "RATE: $tax_rate";
my $single_quote = 'RATE: $tax_rate';

print "Content-type: text/plain\n\n";
print "DOUBLE: $double_quote | SINGLE: $single_quote\n";
```

Question 75: Tigo Pesa Control Flow Loop

Tracing loop statements processing complex mathematical expressions:

```
#!/usr/bin/perl
use strict;
use warnings;

my @values = (5, 10, 15);
my $result_sum = 0;

foreach my $val (@values) {
    if ($val % 3 == 0) {
        $result_sum += $val * 2;
    } elsif ($val > 8) {
        $result_sum += $val + 1;
    } else {
        $result_sum -= 2;
    }
}
print "Content-type: text/plain\n\n";
print "SUM: $result_sum\n";
```

Question 76: LATRA Route Checker Perl Context Evaluation

Evaluating variables in scalar vs list context across expressions:

```
#!/usr/bin/perl
use strict;
use warnings;

my @active_permits = ("Kibo", "Abood", "Sumry");
my $scalar_context = @active_permits;
my ($list_context) = @active_permits;

print "Content-type: text/plain\n\n";
print "SCALAR: $scalar_context | LIST: $list_context\n";
```

Question 77: NHIF Card Validator Regular Expressions

Using regex patterns to extract and validate matches inside structural strings:

```perl
#!/usr/bin/perl
use strict;
use warnings;

my $card_id = "NHIF-9988-TZ";
my $matched = ($card_id =~ m/^NHIF-(\d+)-TZ$/) ? "Valid" : "Invalid";
my $captured = $1 || "None";

$card_id =~ s/TZ/HQ/;

print "Content-type: text/plain\n\n";
print "MATCH: $matched | CAPTURED: $captured | MODIFIED: $card_id\n";
```

Question 78: CRDB ATM Core Subroutines

Evaluating arguments unpacking within legacy subroutine stacks:

```perl
#!/usr/bin/perl
use strict;
use warnings;

sub process_deposit {
    my ($balance, $amount) = @_;
    $balance += $amount;
    return $balance;
}

my $initial = 50000;
my $updated = process_deposit($initial, 15000);

print "Content-type: text/plain\n\n";
print "INITIAL: $initial | UPDATED: $updated\n";
```

Question 79: HESLB Batch Range Operations

Evaluating range loops and modifications to list values in Perl:

```perl
#!/usr/bin/perl
use strict;
use warnings;

my @range_values = (1..4);
my @selected_slice = @range_values[1..2];
my $output_vals = join("-", @selected_slice);

print "Content-type: text/plain\n\n";
print "SLICE: $output_vals\n";
```

Question 80: Mzumbe Portal Legacy HTML Generator

Analyzing a CGI output stream that constructs structural elements of an XHTML view:

```perl
#!/usr/bin/perl
use strict;
use warnings;
```

```
4
5  my $title = "Mzumbe Portal";
6  my $user = "Bertha";
7
8  print "Content-type: text/html\n\n";
9  print "<html><head><title>$title</title></head>";
10 print "<body><h1>Welcome, $user!</h1></body></html>\n";
```